



WEB SERVICES et INFONUAGIQUE

Youssou KASSE

Université Alioune DIOP de BAMBEY

youssou.kasse@uadb.edu.sn

MASTER 2 SI

2021 2022

Json (JavaScript Object Notation)

- Est un format de représentation des données comme XML;
- Fréquemment utilisés dans les API et les fichiers de configuration;
- Il est léger et facile à lire/écrire;
- Facilement intégrable dans plusieurs langages de programmation.

Json: Type de données supportés (.json)

- Json définit 6 types de données:

- **String:**

```
{  
  "name": "Alex"  
}
```

- **Nombre:**

```
{  
  "age": 24  
}
```

Json: Type de données supportés (.json)

- **Objet:**

```
{ "person": { "name": "Alex" } }
```

- **Tableau:**

```
{ "tab": [1, 2, 3, 4, 5, 6] }
```

- **Boolean:**

```
{ "open": true, "close": false }
```

- **Date:**

```
{ "DateNaissance": "1996-01-25" }
```

Exemple de fichier xxx.json (<http://jsonviewer.stack.hu/>)

```
{
```

```
  "fruits":
```

```
    [ { "kiwis": 3,  
      "mangues": 4,  
      "pommes": null },  
      { "panier": true } ],
```

```
  "legumes":
```

```
    { "patates": "amandine",  
      "poireaux": false },
```

```
  "viandes" :["poisson","poulet","boeuf"]
```

```
}
```

Exemple de lecture

- Exemple d'objet Json:

```
var person = { "name" : »Moussa", "age" :  
"24", "address" : "Bambey" };
```

- Pour accéder aux champs, nous pouvons faire dans un programme javascript:

```
document.writeln("Le nom est: " +person.name);  
document.writeln("L'age est: " + person.age);  
document.writeln("L'adresse est: "+ person.address);
```

Exemple de lecture

- Dans un tableau contenant plusieurs objets, nous accédons aux champs avec:

```
var persons = [  
  { "name" : "Moussa", "age" : "24", "address" : "Paris" },  
  { "name" : "Aida", "age" : "22", "address" : "Thies" }  
];
```

- Pour y accéder:

```
document.writeln("Le nom de la 1ère personne est: "  
+person[0].name); //Moussa  
document.writeln("Le nom de la 2ème personne est: "  
+ person[1].name); //Aida
```



Architecture logicielle

INTRODUCTION

- Depuis toujours les systèmes d'information ont cherché à offrir un accès intégré et généralisé à l'information pertinente.
- Ceci s'est traduit ces dernières années par un couplage en interne aux technologies de l'intranet et en externe, à celles de l'Internet, particulièrement dans le cadre du Web

Exigences d'un projet informatique

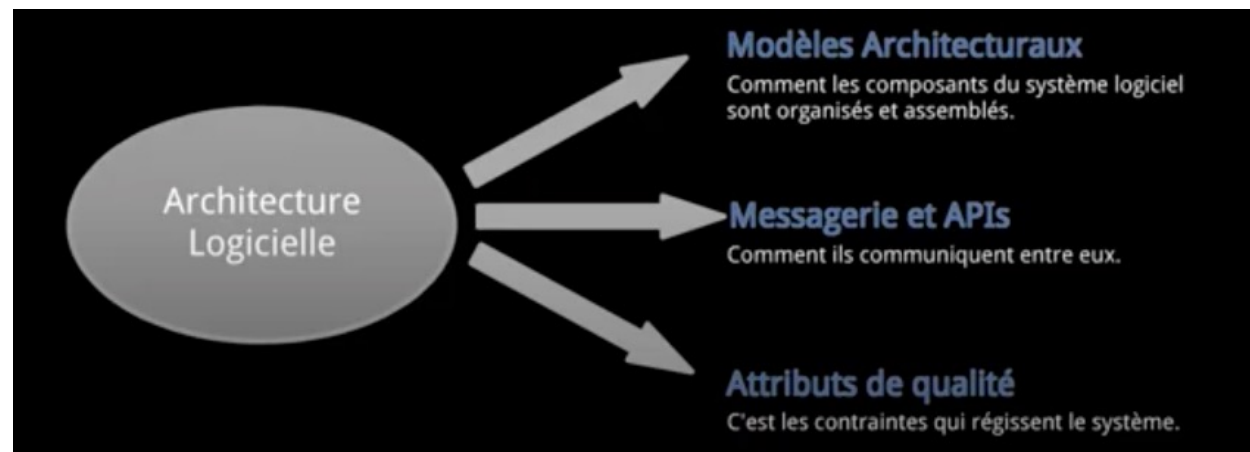
- Exigences fonctionnelles:
 - Une application est créée pour répondre, tout d'abords aux besoins fonctionnels des entreprises
- Exigences Techniques:
 - Performances:
 - Temps de réponse;
 - Haute disponibilité et tolérance aux pannes;
 - Éviter le problème de montée en charge.

Exigences d'un projet informatique

- La maintenance
 - Une application doit évoluer dans le temps;
 - Elle doit être extensible.
- Sécurité
- Portabilité
- Distribuée
- Capacité à pouvoir communiquer avec d'autres applications distantes
- Capacité à fournir le service à différents types de clients (Mobiles, Desktop ...

Architectures logicielle (Définition)

- C'est comment les composants d'un système logiciel sont organisés et assemblés;
- Comment ils communiquent entre eux;
- Quelles sont les contraintes qui régissent le système.





Architectures logicielles (But)

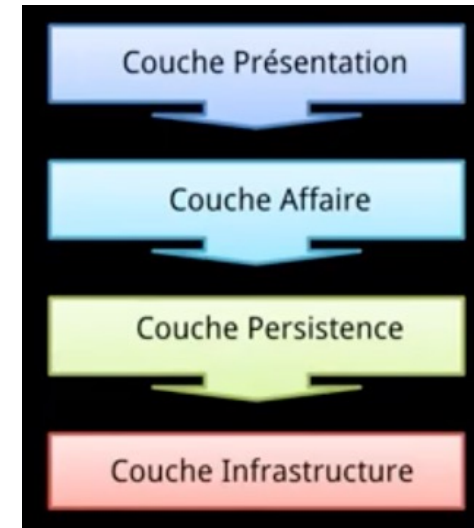
- Faciliter le développement, l'évolution, le déploiement et la maintenance du système;
- Minimiser le temps et le coût d'intervention;
- Maximiser et maintenir la productivité des développeurs face aux changements;
- Rendre les modification simples et rapides

Architectures logicielles (Attention !!!)

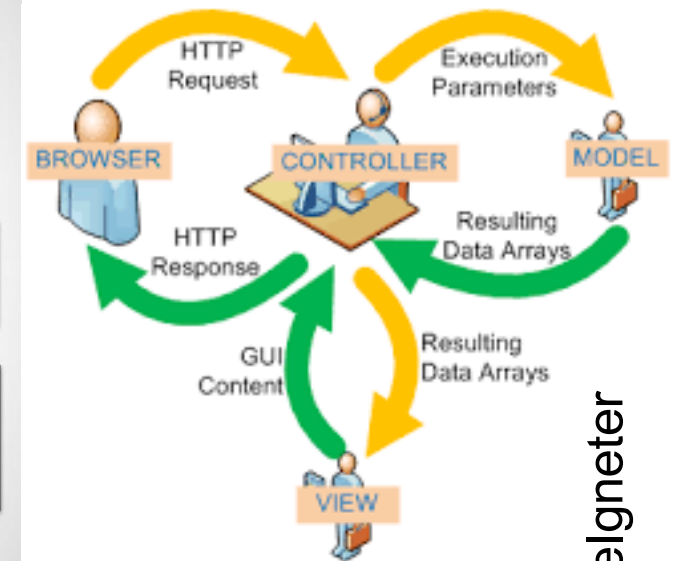
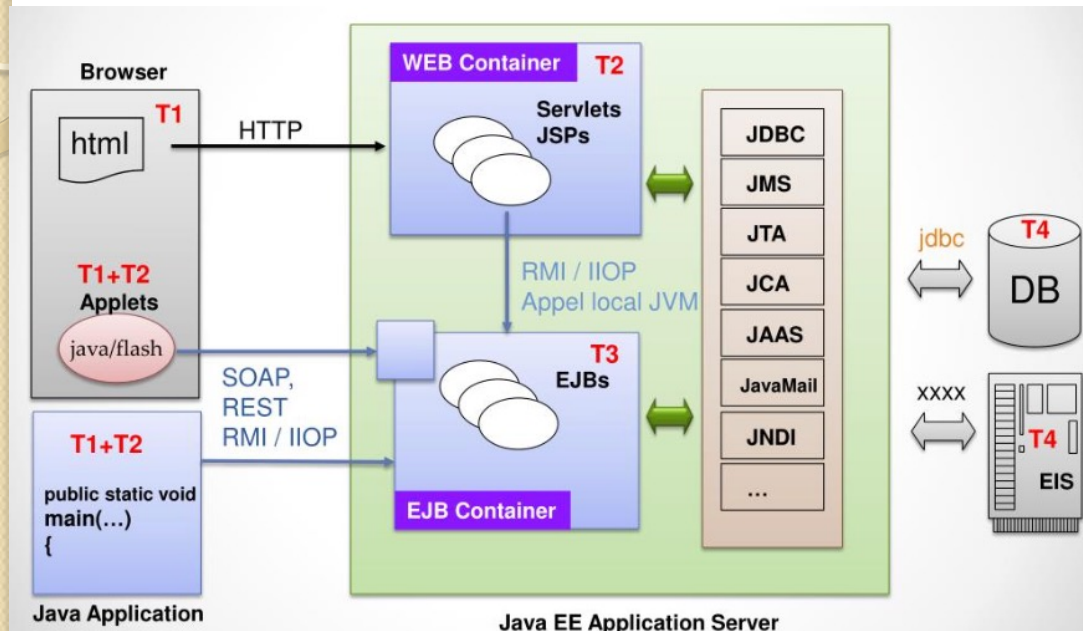
- Une bonne architecture logicielle permet de faire fonctionner correctement un logiciel (**Pas l'objectif**);
- On peut disposer d'un produit qui marche sans architecture logiciel, ou non compris par le développeur (framework) ou avec une mauvaise architecture;
- C'est le cas de la plus part des logiciels en entreprise (**Exo: Proposer l'architecture d'une application que vous avez déjà développée???**);
- Mais le prix à payer pour le développeur est que l'évolution de son application peut être un véritable casse tête (**objectif visé**).

Architectures logicielles: architecture en couches

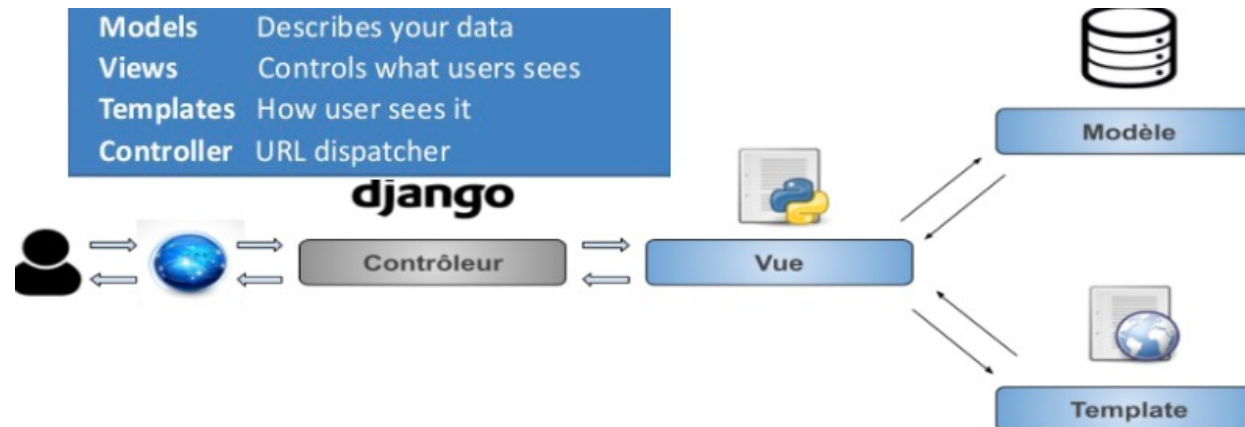
- Plus connue sous le modèle N-tier;
- Monolithique (un seul gros bloc);
- Organisé en couche horizontale (pas de hiérarchie);
- Composant interconnectés, mais pas de dépendance;
- Méthode traditionnelle de conception de la plus part des logiciels en entreprise.



Architecture en couches: Exemple de Framework



CodeIgneter



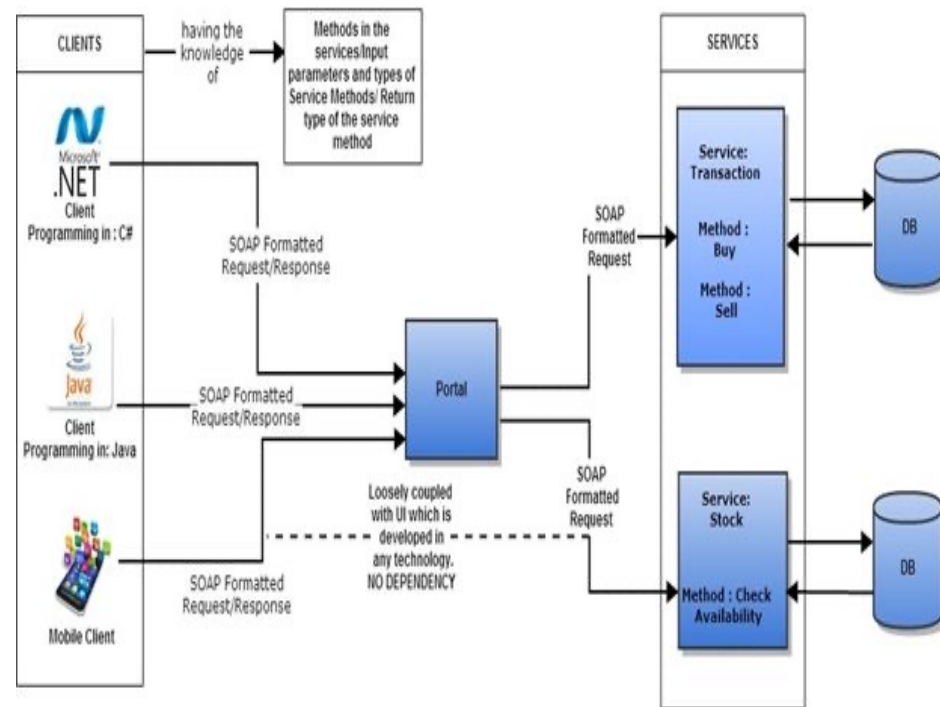
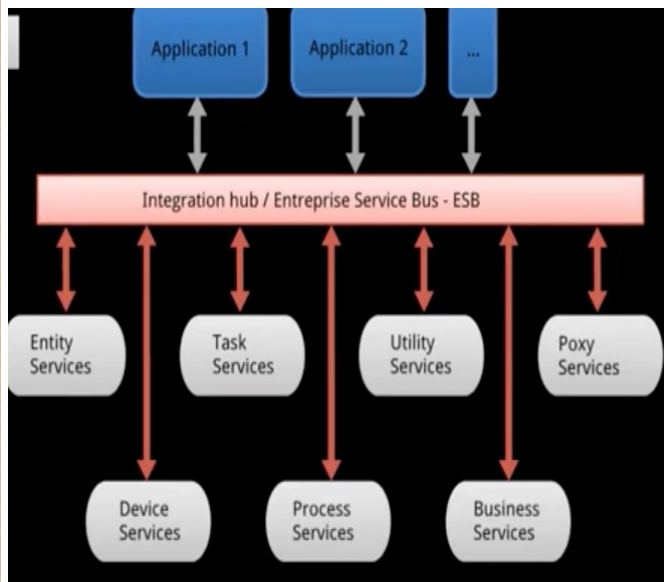


Architectures logicielles: architecture orienté service (SOA)

- Décrit un ensemble de composants distribués qui fournissent et/ou consomment des services.
- Les composants peuvent utiliser différents langages et plateformes d'implémentation;
- Les services sont autonomes et à grosses granularités;

Architectures logicielles: architecture orienté service (SOA)

- Les fournisseurs et les consommateurs sont généralement déployés de manière indépendante.



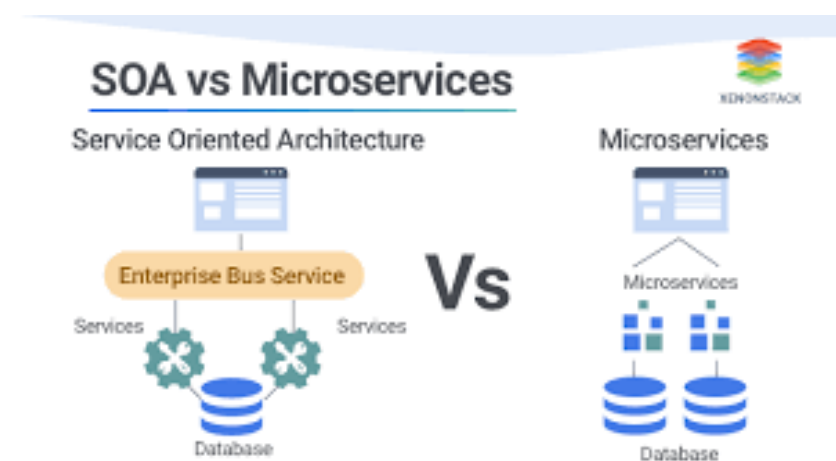
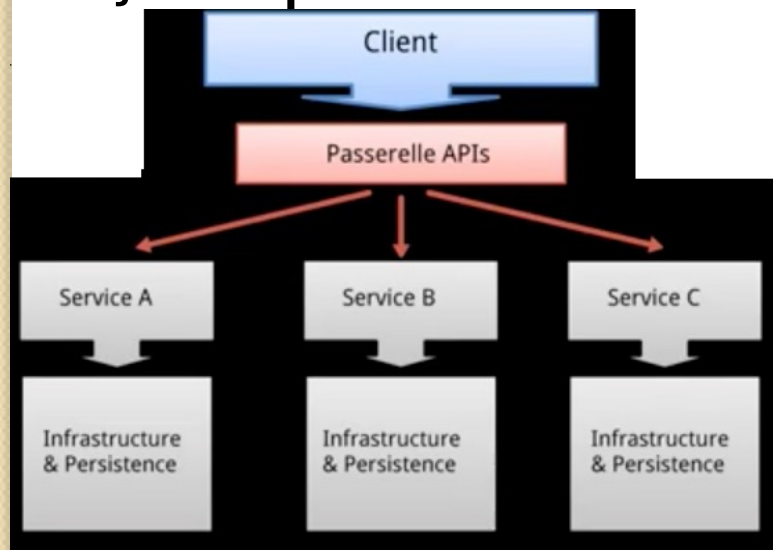


Architectures logicielles: architecture des Microservices

- Modèle basé sur des services (différent de SOA)
- L'application est structurée sous la forme d'un ensemble de services:
 - Autonomes, testables, hautement maintenables, pas de couplage, déployés de façon indépendante.
- Les services sont organisés autour des compétences métier

Architectures logicielles: architecture des micro services

- Les services sont administrés par de petites équipes;
- Permet la livraison rapide, fréquente et fiable de grandes applications complexes (Uber, Airbnb, Blablacar ...);
- Permet de faire évoluer sa pile technologique de façon optimale.





Les services Web



HISTORIQUE

- l'idée de fournir un service informatique centralisé remonte aux années 1960,
- Lorsque des services informatiques étaient fournis sur un réseau et utilisant la technologie « time-sharing » sur les mainframes.



HISTORIQUE

- Dans les années 80, les entreprises ont réalisé que les serveurs basés sur des ordinateurs normaux pourraient être installés à moindre coût à la place des mainframes
- Progressivement, ces mainframes ont été remplacés par des ordinateurs normaux en raison aussi de la baisse des prix des ordinateurs personnels



HISTORIQUE

- Dans les années 90, avec le début de la globalisation de l'internet, la mode qui consiste à plusieurs ordinateurs d'accéder à un grand serveur refait surface.
- Depuis ce temps, de plus en plus de services sont offerts sur internet et les utilisateurs de ces services demandent plus de capacité de stockage.

HISTORIQUE

- Fin des années **quatre-vingt-dix**, une période où de nombreuses nouvelles technologies font leur apparition, comme par exemple XML
- Facteurs majeurs, la démocratisation des réseaux et d'Internet en particulier.
- Avant 1995-1998, un ordinateur était principalement destiné à fonctionner de façon autonome.

HISTORIQUE

- Aujourd'hui, un ordinateur sans accès au réseau des réseaux (Internet) se trouve limité (fonctionnalités, informations).
- d'où une évolution et un changement totalement radical de l'utilisation de l'ordinateur.
- Mais ceci va poser un certain nombre de problèmes

HISTORIQUE

- Un problème de gestion du parc d'ordinateurs hétérogène,
- Un problème d'**interopérabilité** et de **connectivité** des utilisateurs, systèmes, applications, périphériques se trouvant dans le périmètre de l'entreprise, mais aussi chez les constituants externes : partenaires, clients et fournisseurs.

HISTORIQUE

- Nombreuses méthodes et technologies ont été utilisées pour résoudre ces problèmes
- CORBA avec IIOP (OMG)
- ***Distributed Component Object Model***
DCOM (Microsoft)
- RMI (Sun)
- Ils ont connu un succès sur à l'intérieur de l'entreprise

Historique: En bref

- En 1980, le RPC est défini pour faciliter les communications entre les clients et les serveurs.
- C'est un principe qui appelle une procédure externe dans une autre zone mémoire.
- Pour cela, votre collaborateur doit définir ce qu'il a à vous offrir (IDL) sous forme de procédures.
- Par exemple Java-RMI: ici le serveur expose des méthodes avec les paramètres d'appel.

HISTORIQUE : inconvénients

- Mais, Ces différentes approches présentent de nombreux inconvénients
- Les protocoles ne passent généralement pas les firewalls,
- La même technologie doit être déployée de chaque côté pour assurer la communication,
- Elles sont complexes car le développeur doit avoir une connaissance de l'application à distance (méthodes, paramètres) avant d'écrire le code pour l'utiliser,

HISTORIQUE: inconvénients

- Elles sont fragiles car si le partenaire fait évoluer son application, le code client peut ne plus fonctionner.
- Les services Web apparaîtront comme solution conçue pour faciliter les échanges de données, mais aussi l'accès aux applications au sein des entreprises et surtout entre les entreprises.

INTRODUCTION

- Les services Web sont une nouvelle manifestation de cet objectif
 - afin de permettre la réalisation rapide et efficace de systèmes d'information distribués sur Internet ou intranet
 - en intégrant des applications existantes et nouvelles
- Qu'est ce qu'un service Web et pourquoi un tel engouement pour cette technologie ?

DEFINITION

- Un service web est un logiciel identifié par une URI dont l'interface est publique et les liaisons sont définies et décrites en utilisant XML
- Un service Web est aussi défini comme un ensemble de protocoles et de normes utilisés pour échanger des données entre les applications.

DEFINITION

- Mais pour résumer on peut simplement dire qu'un service Web est un moyen d'accéder à un service par l'intermédiaire du Web.

Historique: En bref

- En 1998, SOAP a été défini.
- Le client et le serveur échange en utilisant uniquement des messages (XML uniquement)
- Fonctionne avec n'importe quel protocole (SMTP, FTP, HTTP ...)
- L'IDL utilisé est du WSDL
- Même si on continue de l'utiliser en industrie, le groupe en charge de la spécification ne travaille plus (donc déconseillé pour un nouveau projet).

Exemple de services web

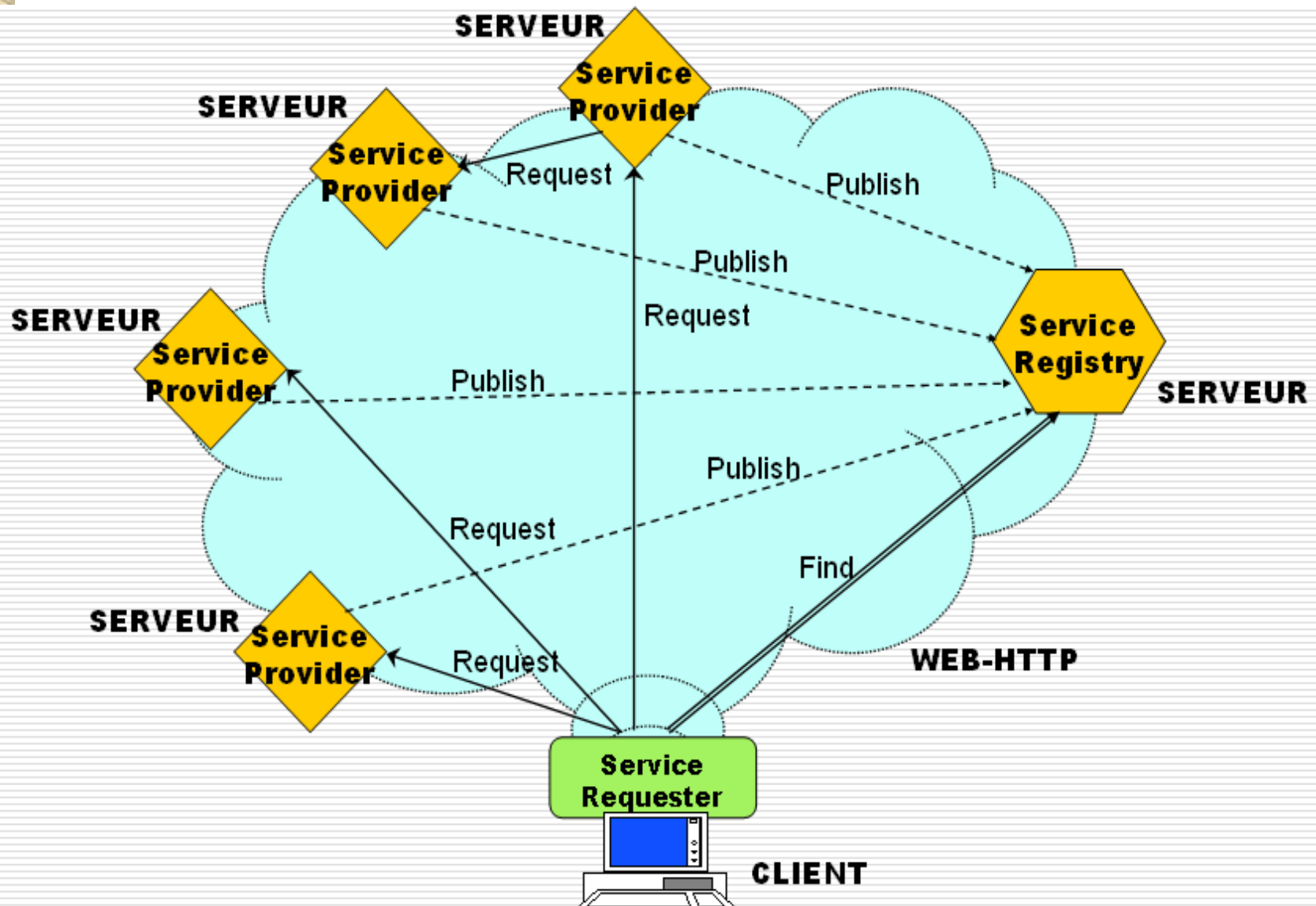
- Google (<http://www.google.com/apis/>) :
 - accès gratuit mais limité (1000 requêtes par jour après enregistrement)
- Amazon (<http://aws.amazon.com/fr/>)
 - accès gratuit mais limité (1 requête par seconde après enregistrement)



Exemple de services existants

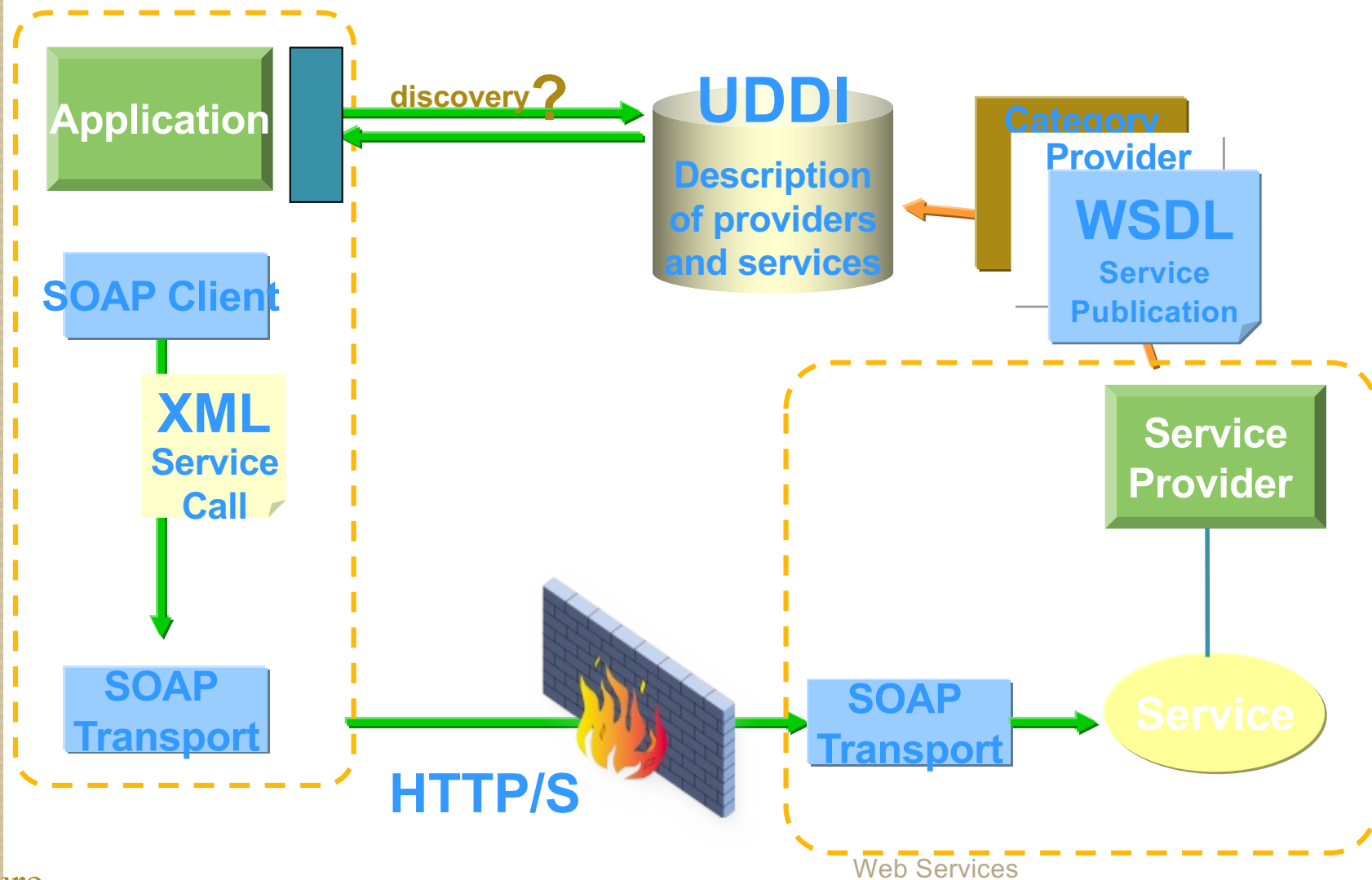
- Outils vous permettant de vous organiser, de communiquer et de planifier votre travail: Google Docs, SlideRocket, Toggl (Free Time Tracking Software), etc...
- Réduction d'URL (goo.gl, TinyURL, bit.ly, etc.)
- De nombreux services proposés par Google <https://developers.google.com/apis-explorer/#p/>
 - Analyse de données
 - Traduction
 - Réduction d'URL
 - Statistique fréquentation site web
 - etc.

III.2 ARCHITECTURE



Modèle fonctionnel

(1/1)





III. 4 SOAP

(Simple Object Access Protocol)

- Un protocole de communication entre applications fondé sur XML,
- Un protocole indépendant de toute plateforme et de tout langage de programmation.
- Permet de contourner les firewalls
- Est une recommandation W3C depuis juin 2003 (SOAP 1.2)
- S'appuie sur HTTP, SMTP, FTP ...



WSDL (*Web Services Description Language*)

- créé dans le but de fournir une description unifiée des services web
- Un fichier WSDL est un fichier tout simple, plus précisément, c'est un fichier XML
- Un fichier WSDL contient une description de tout ce qui est nécessaire à l'appel d'un service Web :

WSDL (*Web Services Description Language*)

- L'URL et l'espace de noms du service
- Le type de service Web (en général des appels de fonctions par SOAP)
- La liste des fonctions disponibles
- Les arguments de chaque fonction
- Le type de données de chaque argument
- La valeur de retour de chaque fonction et le type de données de chaque valeur de retour

WSDL (structure)





UDDI (*Universal Description, Discovery and Integration*)

- Il fournit un protocole, une API et une plateforme permettant aux utilisateurs de services web, depuis n'importe quel système, de localiser dynamiquement à travers Internet les services web qu'ils désirent utiliser.
- UDDI permet aux fournisseurs de services de s'inscrire et de répertorier les services qu'ils proposent

UDDI

- UDDI ne contient que des références associées à des services, et non les services eux-mêmes
- Il peut être public ou privé.



API REST

Historique: En bref

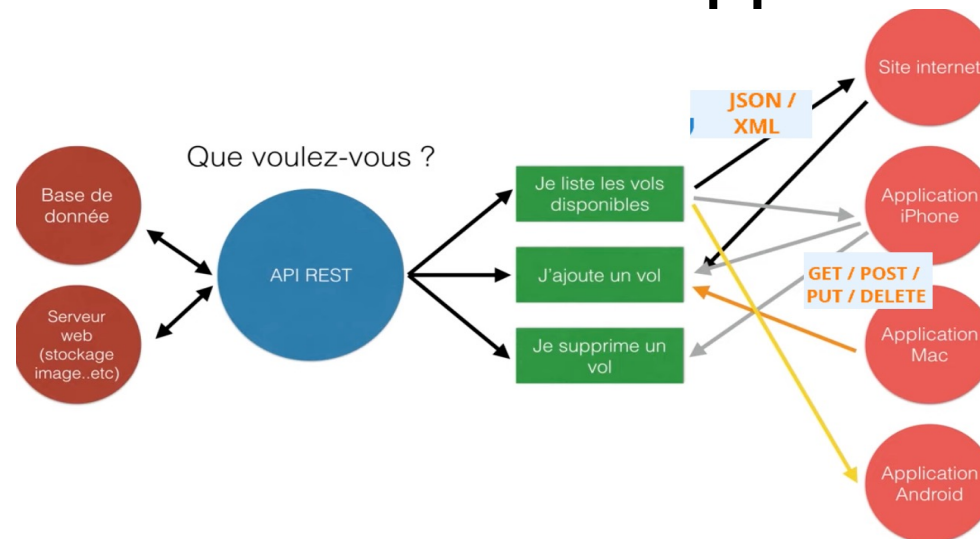
- A la suite de SOAP, nous avons REST
- Une API REST est une interface entre le client et le serveur dans le but de récupérer des données.
- Pour utiliser une API REST, on utilise une URL couplée à une requête HTTP.
- Exemple: Derrière une requête google maps, Google utilise par exemple la requête suivante pour trouver une adresse (ici Sandaga):
<https://maps.googleapis.com/maps/api/geocode/json?address=sandaga+dakar>

API REST

- **REST** (pour *REpresentational State Transfer*) est une **type d'architecture d'API** qui fournit un certain nombre de **normes** et de **conventions** à **respecter** pour **faciliter** la **communication entre applications**.
- Les APIs qui respectent le standard REST sont appelées **API REST** ou **API RESTful**.

API REST

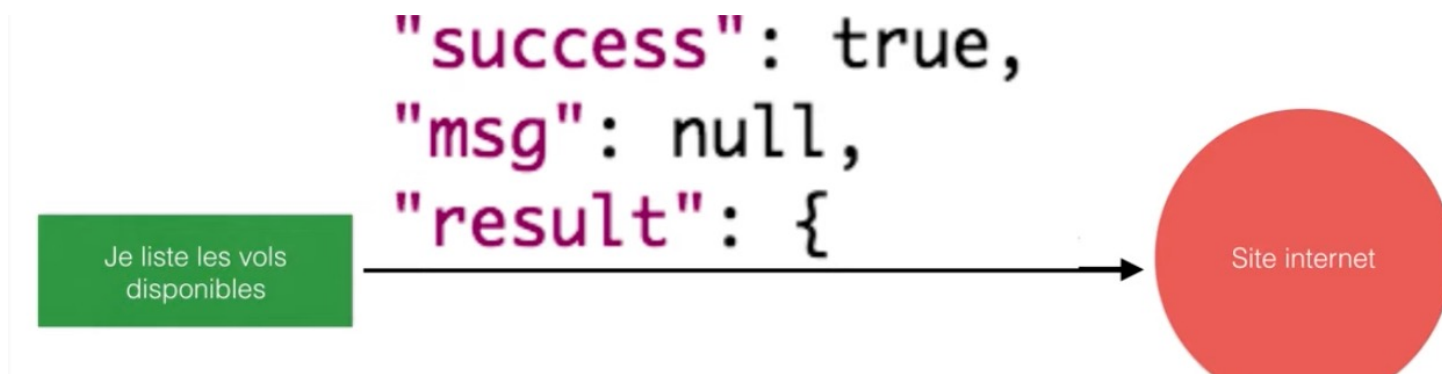
- Une API est interface permettant l'interaction entre différentes applications..



- Concevoir, développer ou maintenir une **API REST** sont aujourd'hui parmi les missions les plus communes pour un développeur **back-end** ou **fullstack**.

API REST

- Si nous zoomons sur la requête : « liste des vols disponibles » et que l'on considère qu'il n'y a pas de message d'erreur
- Le résultat de l'application REST ressemble à:



API REST

- Plus en détail, le fragment résultat ressemble à:
- Le développeur front n'a qu'à exploiter ce résultat à travers son application web, mobile ...

```
"result": {  
  "nombre": 4,  
  "vols": [  
    {  
      "0": "5",  
      "1": "Paris",  
      "2": "Toulouse",  
      "3": "2017-05-24 16:20:00",  
      "4": "1",  
      "5": "0",  
      "id": "5",  
      "ville_depart": "Paris",  
      "ville_destination": "Toulouse",  
      "date_depart": "2017-05-24 16:20:00",  
      "nb_heure_vol": "1",  
      "prix": "0"  
    },  
    {  
      "0": "23",  
      "1": "Paris",  
      "2": "Martinique",  
      "3": "2017-06-22 10:00:00",  
      "4": "13",  
      "5": "800"
```



API REST

- En se positionnant en tant que développeur backend, mettez en place une API REST en utilisant PHP
- Vous pouvez partir d'une base de données de votre choix avec une seule table.

API: Exemple

- Requête:

<https://fr.wikipedia.org/w/api.php?action=query&prop=extracts&exchars=500&explaintext&titles=mars>

```
{
  "batchcomplete": "",
  "query": {
    "normalized": [
      {
        "from": "mars",
        "to": "Mars"
      }
    ],
    "pages": {
      "1962": {
        "pageid": 1962,
        "ns": 0,
        "title": "Mars",
        "extract": "Mars est originellement le nom du dieu de la guerre dans la mythologie romaine. De cela proviennent plusieurs noms :\n\nplan\u00e9te Mars, quatri\u00e8me plan\u00e9te du syst\u00e8me solaire.\nmois de mars, troisi\u00e8me mois de l'ann\u00e9e dans les calendriers occidentaux.Mars peut aussi faire r\u00e9f\u00e9rence \u00e0 :\n\n\n== Anthroponyme ==\n\n\n== Pr\u00e9nom ==\nMars de Nantes (Vie si\u00e8cle), \u00e9v\u00eaque de Nantes ;\nMars de Royat ou Martius (c. 527), ermite puis abb\u00e9 \u00e0 Royat ;\nMars Di Bartolomeo (n\u00e9 en 1952), homme politique luxembourgeois,\nMars Simard, fondateur..."
      }
    }
  }
}
```

- Ces résultats sous format json sont par la suite exploités par des application (avec js ou autres).

Web Service et API

- Les API sont des services exposés par des entreprises pour faciliter le travail du développeur.
- Ils sont généralement intégrés sur la machine hôte contrairement aux web services.
- Ils lui évitent de réinventer la roue.

